UNIVERSITY OF COPENHAGEN

Bachelor Thesis

The Shortest Path Problem In Mazes

Oliver, GBT989
Supervisor: Jacob Holm

Datalogisk Institut
Københavns Universitet

January 10th, 2020

Abstract

In computer science, the shortest path problem has been a classical issue that has been examined by mathematicians and computer scientist alike. In this thesis, we examine 5 different algorithms that provide either a shortest path - or an estimated shortest path.

These 5 algorithms consist of a Breadth-First Search, Dijkstra's Algorithm, A* Shortest Path, HPA* Shortest Path and finally Jump Point Search.

We examine this by implementing the five algorithms examined in C++ and then tests the runtime, memory usage and accuracy of their shortest path solution.

Of the estimated shortest path algorithms, it was found that Breadth-first search could provide shortest paths solutions within 1% of the optimal solution. It was found that HPA* ranged between 3.77% to 13.82% and concluded that this was due to the fine-tuning of the cluster size variable not being properly handled.

Of the optimal shortest path algorithms, it was found that Dijkstra ran in computational time and memory usage equal to- or a bit worse than that of Breadth-first search. We found that A* provided, in some cases, a significant speed- and memory usage improvement.

Lastly, it was found that Jump Point Search was the fastest algorithm and the best in terms of memory usage, while still being accurate.

In the end, this thesis addressed future improvements to tests and implementations, such as the fine-tuning of HPA* to get more accurate results and the translation of abstract game maps into grids.

9th of May 2021 - I did this paper back in 2020 and I have made some small corrections throughout that is marked in blue text to ensure correctness.

During my verbal defence, the censor got curious as to why I had written Djikstra's running time as O(n^2). This side-track ended up in a confused me, trying to explain my reasoning behind, but failing to do so, with a combination of nerves, confusion and well. Yuck - verbal exams can be a pain.

I still stand by my report and it is one of my proudest accomplishments during my time at Copenhagen University.

The final grade of the project after the defence was 4.

Contents

1 Introduction **3**

2 Theory **4**
2.1 History of the Shortest Path Problem 4
2.2 The Graph as a Grid 5
2.3 BFS and Dijsktra Shortest Path 6
2.3.1 Breadth-first Search 6
2.3.2 Dijsktras Algorithm 7
2.4 A* Shortest Path 9
2.4.1 Choosing a Heuristic Function 10
2.5 HPA* Shortest Path 12
2.6 Jump Point Search Shortest Path 15
2.7 Other notable algorithms 17

3 Implementation **19**
3.1 Tools Used 19
3.2 Creating the Tests 19
3.3 Measuring the algorithms 20

4 Results **22**
4.1 Test 1 23
4.2 Test 2 24
4.3 Test 3 26
4.4 Test 4 27
4.5 Test 5 29
4.6 HPA* Preprocessing 30

5 Discussion **31**
5.1 In General 31
5.2 Breadth-first, Dijkstra and A* 31
5.3 HPA* 32
5.4 Jump Point Search 33

6 Conclusion **35**

A Bibliography **36**

B Results **38**
B.1 Runtime 38
B.2 Memory 40
B.3 Path lengths 42

C Code **44**

1 Introduction

The shortest path problem is a tale as old as time. Humans have always strived after being as cost-efficient as possible. Minimizing resource-cost such as distance travelled, time, and more have been an intuitive concern since the middle ages and probably further back. Finding the shortest path between two villages to transport goods could optimize the yield from trades that would find place. Tales of a labyrinth containing a Minotaur is mentioned in Greek mythology. In the era of a world that is becoming more and more digitized, new mazes appear, that we now may use computers to solve for us. Going from one end of the city to the other in the fastest way possible is now as simple as typing the addresses on a phone and following the result.

This thesis will focus on five different algorithms and their usage on specifically grid-based maps, by using pictures as the main data structure. This is done by implementing the five algorithms chosen in a programming language called C++ and testing their performance against one another in solving the same problem.

Understanding these algorithms is key to understanding how a data structure can affect- and be affected by different methods and how each of these may provide benefits or drawbacks to solving the problems at hand.

A perfect solution is not always the best to use is practice. Sometimes, it may yield better results to take a suboptimal solution rather than taking the perfect one. This is why not all algorithms necessarily return the shortest path, but an alternative path that may be taken instead.

Chosen amongst the pathfinding algorithms are Breadth-First search and HPA*, two algorithms that return a suboptimal path. Amongst the guaranteed shortest path algorithms chosen are Dijkstra's Algorithm, A* and Jump Point Search. Each of these guarantees to always find the shortest path.

This thesis is structured this that Chapter 2 gives an overview of the shortest path problem historically and theoretically for each algorithm. Chapter 3 describes the implementation steps for the algorithms taken for the examinations and describes the creation of the tests to be examined. Chapter 4 then presents the results of these tests and Chapter 5 discusses the finds, optimizations and errors that may have occurred or impacted the results.

2 Theory

2.1 History of the Shortest Path Problem

Typically, we see that the shortest path problem is historically connected with finding the shortest path through a maze. (Tarry 1895) However, it is not known when exactly standardized procedures for finding the shortest paths originated. `Moore` claims that looking for these specific solutions is a twentieth-century issue. (Moore 1957) However, the paper by `Tarry` from 1895 already examines the issue of finding a solution in the form of a path through a maze.

Non-surprisingly, this field of study is one that mathematicians have described and examined, as we have already seen. More specifically, most people would mention the swiss mathematician `Leonhard Euler` as one of the founding fathers of graph theory, a core mathematical subject in finding the shortest path. In 1736 he released a paper, based on a talk in 1735, about an old puzzle where one was presented with seven bridges that connected a set of islands. The goal was to cross each bridge once - without ever visiting the same bridge twice. (Biggs 1986, cop. 1976, pp. 3-11). Essentially, he proved there was no solution with the help of what is a simple form of graph theory. It is important to note, that Euler *did not* present the situation as a graph. However, he represents the bridges as letters a, b, c, d... and the islands after capital letters A, B, C, D... And as such, the groundwork for graph theory was formed.

It would not be until 1936 where the Hungarian mathematician `Dénes König` caused a breakthrough in the history of graph theory in his *Theorie Der Endlichen Und Unendlichen Graphen* book. (Koenig 1990) Here he managed to describe all the basic parts of graph theory, which could later be used to present shortest path problems.

It is not all mathematicians such as `Moore` that concern themselves with the history of this problem, unfortunately. `E.W. Dijkstra`, whose paper described a solution to finding the shortest path in a minimum spanning tree, gives us very few clues as to the background for the problem. Rather, he focuses on describing the problem at hand and presenting a solution. (Dijkstra 1959)

`Dijkstra` is one of those people who does not concern himself with the running time of his algorithm. A lot of the authors to these early algorithms does not use running time as a way to describe the efficiency of their algorithms. `Moore`, however, did describe both memory usage and complexity of his solutions, even providing an alternative algorithm with the intention of reducing memory usage. (Moore 1957), whereas `Dijkstra` only showed concern as to whether his method was preferable to that of other current solutions. (Dijkstra 1959)

Having the groundwork for graph theory is important for this problem. The reason as to why is because we may translate a lot of problems from complex situations into a common set of subproblems in the forms of graphs. This means that a lot of algorithms continue to work on these problems, even though they initially were not meant for such a problem. For example, we may represent a road network as a graph, and use Dijkstra's algorithm to find a path

from a source v to a destination u. It is important to note, that Dijkstra's algorithm only works for *positive* weighted graphs (where each step increases the total weight).

A negative-weighted graph would have to use an algorithm such as Bellman-Ford. This algorithm, a combination of a paper released by `Richard Bellman` (Bellman 1958) and the book *Flows in Networks* (Ford and Fulkerson 1962), can traverse a graph with negative weights, although its worst-case runtime is worse than that of e.g. Dijkstra's Algorithm. `Ford` describes a good example of how graphs at the time had come to be accepted as the denominator in these kinds of problems and were starting to be used by scientists in their descriptions of this kind of problem. (ibid.)

While working on graphs, finding the shortest path can be divided up into 4 subproblems:

- **Single-source shortest path problem:** Given a vertex u find the shortest path to all other vertices.
- **Single-destination shortest path problem:** Given a destination vertex v, find the shortest path form all other vertices to this. This can be made into a single-source shortest path problem, by reversing the direction of the edges.
- **Single-pair shortest path problem:** Given a pair of vertices u, v find the shortest path between these.
- **All-pairs shortest path problem:** For all pairs of vertices u, v find the shortest path from u to v.

In this thesis, we will be examining the *single-pair shortest path problem*, since we are interested in finding the shortest path between our source and a goal to find our way through mazes of different forms.

2.2 The Graph as a Grid

In this thesis, we have so far touched the subject of graph theory and how it is related to finding the shortest path. Specifically, this paper examines how one may find the shortest path in a grid-based world.

Our world is entirely 2-dimensional. Each tile in the world is made of two values, an x and y coordinate. We can represent this visually by using the same structure that a picture uses. A picture has an x-axis and a y-axis. Each tile on the picture is called a pixel and is made of two coordinates x and y. Furthermore, each pixel contains a colour-value, defined by a set of 3 values between 0 and 255, defining the intensity of each of these colours. The colours are, in the following order: Red, green and blue.

By using these colour values, we can define a pixel as a solid wall, having the colour black: $\{0, 0, 0\}$ or as a walkable tile, having the colour white: $\{255, 255, 255\}$. Furthermore, we can tell our program that we want to start at a green node $\{0, 255, 0\}$, and end at a blue node $\{0, 0, 255\}$.

Moving horizontally or vertically costs exactly 1 while moving diagonally costs $\sqrt{2}$. This is because that the euclidean distance of $\sqrt{1^2 + 1^2} = \sqrt{2}$. It is,

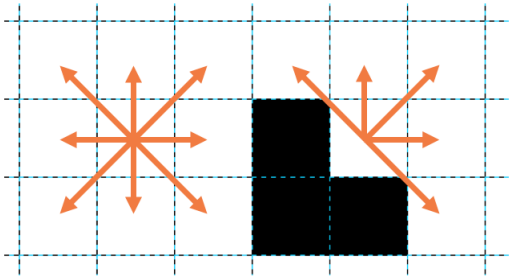

Figure 1: The movements that can be taken from a single pixel. Each arrow represents an edge from the originating vertex.

therefore, more optimal to move diagonally than exclusively moving horizontally or vertically. While we, in theory, could limit our graph to such movements, we would not be able to use the Jump Point Search algorithm, as this relies on being able to move diagonally.

We can simplify the pixel into a vertex on the graph by defining that a pixel is itself a vertex if it is walkable. If its neighbour is a walkable pixel, we can connect the two vertices by an edge with a weight of either 1 or $\sqrt{2}$ depending on whether the pixel is diagonally placed or not. Thereby, we have created a graph, that we can explore through different methods.

2.3 BFS and Dijsktra Shortest Path

While we have already established that we are working on a weighted graph, I found it interesting to examine an algorithm such as breadth-first search, as it does provide an answer to the shortest path in an unweighted graph. Specifically for BFS, it would see every weight as equal to 1. While this means, that we will not always get an optimal shortest path solution, we can e.g. still find a solution to a maze. Furthermore, Dijkstra's Algorithm uses ideas similar to those in breadth-first search. This will make it interesting to compare, to discern how examining weights may increase running time costs and if it would be worth taking a sub-optimal solution such as BFS.

2.3.1 Breadth-first Search

Breadth-first search was discovered by `Moore`, in the context of routing traffic through a network of telephones or cities. `Moore` himself specifically mentions that all of his algorithms are applicable to finding solutions in mazes. This algorithm is described as Algorithm A in his paper. (Moore 1957, pp. 286–287)

The algorithm itself is relatively simple. It is important to note, that this algorithm does not work in a weighted graph. However, given our grid structure, and our goal of pathfinding through a maze, the algorithm has a use. The path provided may not be the shortest path but it will be a solution if such exists.

This algorithm works by starting at a given start node s, from which it adds every neighbour to the back of the queue. Each neighbour is then set to have

come from the current node we are examining. If any of the neighbours are our goal, we terminate early and follow the path back to our start node s. Otherwise, we continue, examining the next pixel at the front of our queue. (Cormen et al. 2009, p. 595)

As we are interested in the shortest path from our starting vertex s to our goal vertex and not the size or amount of vertices we have to travel, we need only keep track of the previous parent vertex $u.P$. That means, when we hit our goal, we can reconstruct the shortest path to our goal, by following each of the vertices back to the start node. The queue is of a first in-first out type. This ensures, that we are travelling breadth-first.

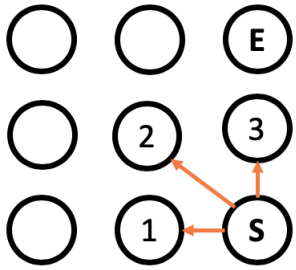

Figure 2: A 3x3 graph using BFS from s

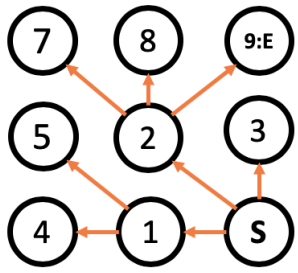

Figure 3: A 3x3 graph using BFS, on the 2nd neighbor.

In Figure 2 we see the first step. We start at s and examines all neighbours, adding them to the queue one by one. Secondly, in Figure 3, we see, that once we reach the 2nd in the queue, we have enqueued every pixel at least once. We have then added all pixels to the queue, but due to the nature of BFS, we may terminate early, before examining the next elements in the queue, once the goal has been added to the queue. In this case, that would mean we could terminate on the second neighbour. This saves some time on the small scale but on a large scale, this time decrease is theoretically irrelevant.

As one may imagine, this algorithm is not very effective. In the worst case, one may end up visiting every pixel. That would make the running time of this algorithm the size of the entire maze $O(n^2)$, plus the time it takes to traverse back the path.

Furthermore, this algorithm isn't accurate. Figure 3 shows an example, where the algorithm will choose a diagonal path with the cost of $2 \cdot \sqrt{2}$, instead of the direct path with a cost of 2. Even without the early termination, once a node has been visited, and its parent has been set, this does not change.

2.3.2 Dijsktras Algorithm

In 1959, the dutch scientist `Edsger W. Dijkstra` released a paper called "*A Note on Two Problems in Connexion with Graphs*", that explained the algorithm that later would be called `Dijkstra's Algorithm`.

In his paper, `Dijkstra` describes two problems. The first is to construct a graph, with a single minimum path connecting each node. Our focus will be

on the second problem, where he talks of finding the total minimum length between two given nodes P and Q. Incidentally, this problem also solves the single-source shortest path problem, even though his solution was presented to solve a single-pair shortest path problem. This algorithm works only on graph with non-negative weights, even though the paper itself does not warn of this. (Dijkstra 1959)

Working through an example, we shall see, how Dijkstra's algorithm works.

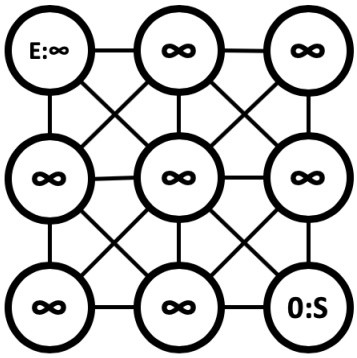

Figure 4: A 3x3 graph using Dijsktra from s. Every black line is an edge.

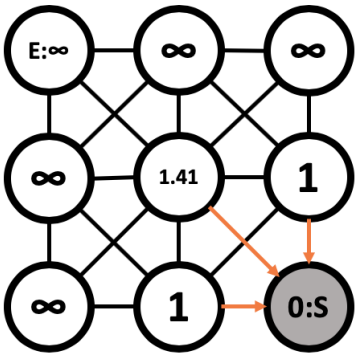

Figure 5: The 1st step in Dijkstras algorithm. The orange arrows represents path to the parent node.

Again, recounting our definition earlier, moving diagonally will have a cost of $\sqrt{2}$ and moving vertically- or horizontally, will have a cost of 1.

Every node is initialised, by having their distance set to ∞. We then start with our first node, s, setting it's weight to 0 and examine each of its neighbours. If it is the first time we see the neighbour, we add the cost of the edge, plus the cost of the node we are examining to the neighbour node. Furthermore, we add the node, that we came from, to our current node. This results in Figure 5. Lastly, we add each neighbour to our queue, sorting such that the lowest weight comes first.

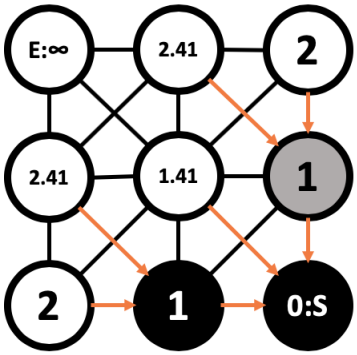

Figure 6: The third step in Dijkstra

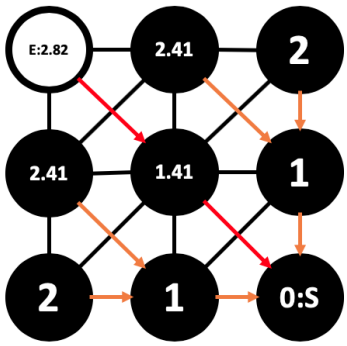

Figure 7: The final step in Dijkstras

Next, the node with the lowest weight is choosen to be examined just as above. If there are two nodes with the same weight, one of these nodes are chosen at random. Once again we examine the edges to the new neighbours of the current node. If a node has already been visited, we observe if this path is shorter than the path already present, by comparing the weights. If so, we replace the old path with the new one. If not, we do nothing. In Figure 6, we can see how black nodes may contain several edges, represented by the black lines. However, each node only has one orange arrow, that represents the shortest path from the current node to our start node s.

Lastly, when we reach our goal node in the queue, we may terminate the algorithm. Due to the nature of us always picking the lowest weight, we have to continue running Dijkstra's Algorithm, until we examine the goal itself, unlike Breadth-First search, where we could terminate early, once we had found it has a neighbour to one of the nodes. Aforementioned way ensures that every other shortest path up until the node has been examined, and that all connecting nodes are connected correctly, as a nodes shortest path can be replaced until it have been examined. This makes it possible to reconstruct the path back to the start node s, as represented in Figure 7

`Dijkstra` does not measure his complexity of his algorithm directly, but is concerned with effectiveness of his algorithm. This results in an algorithm that runs in worst case O(n^2) (I should clarify that n is the number of vertices. It would be more correct to write O($V^2 + E$) = O(V^2) time as well as the time it takes to traverse the path back.

As one may have noticed, this algorithm is close to that of a breadth-first search. The difference lies in Dijkstra's Algorithm being able to work with weights. In our grid-based world, though, this means that the only difference between Dijkstra's Algorithm and a BFS is that the algorithm examines horizontal and vertical neighbours, before examining the diagonal ones.

With Fibonacci Heaps, it managed `Thorup` to modify Dijkstra's algorithm to get a running time of O(E lg lg V), where E are the edges and V are the vertices. (Thorup 2000). It is important to note, however, that these algorithm use an amount of space that depends on the word size of the underlying machine, so the runtime efficiency comes at a trade for more memory usage.

2.4 A* Shortest Path

As we have seen so far, Dijkstra's Algorithm can be very inefficient. In an obstacle-free maze of the size 500 by 500, if the goal and start node are opposite of each other, Dijkstra's Algorithm may have to examine the entire picture, even though no obstacles are blocking the path.

In 1968, `Hart, Nilsson and Raphael` suggested an extension to Dijkstra's algorithm, that would use a heuristic to improve the search. (Hart, Nilsson, and Raphael 1968)

A* seeks to add information from the problem domain to the search. Beforehand, an algorithm such as Dijkstra searches the entire domain without direction. The paper on A* describes how this is wasteful, as we are examining

nodes that we know are not on the optimal path. (Hart, Nilsson, and Raphael 1968, pp. 101–102)

To solve this, they suggest introducing a heuristic function $f(n)$, which is written as the sum of two parts:

$$f(n) = g(n) + h(n)$$

Where $g(n)$ is the cost from our start vertex s up until our current node n, and $h(n)$ is an actual optimal cost from our node n to our goal node g. (ibid., p. 102) As $h(n)$ is a function that is unknown, we choose some function $\hat{h}(n)$ that estimates the length instead.

This can be applied to Dijkstra's algorithm. By changing the weight-function in Dijkstra's algorithm, to prioritise sorting our nodes based of the $f(n)$ function in the queue instead of purely on our weight $g(n)$, we have our A* algorithm.

Picking a good heuristic function is important for the algorithm to work properly. In their discussion, `Hart, Nilsson and Raphael` talk about how making their heuristic function equal zero would be the same as using no information from the problem domain. In our implementation, that would turn the A* algorithm into Dijkstra's algorithm, as our only change to Dijkstra's algorithm is the addition of the $\hat{h}(n)$ function. (ibid., pp. 106–107)

First and foremost, a heuristic function must be admissible. In their paper, they define admissible as our estimate $\hat{h}(n)$ being of a lower bound than $h(n)$. (ibid., pp. 102–103). Furthermore, $\hat{h}(n)$ must be consistent. That is, if there exists an edge between nodes m, n, the heuristic must be less than- or equal to the actual edge. (ibid., p. 104).

The reason for these limitations is clear, in that if $\hat{h}(n)$ would be to overestimate the path, a node may be completely disregarded by the algorithm, in favour for finding another path with values where $\hat{h}(n)$ is smaller. This would result in not the shortest path being found.

2.4.1 Choosing a Heuristic Function

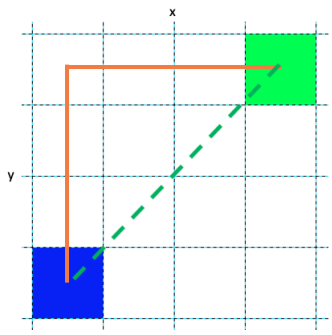

Figure 8: Two heuristic functions. Orange: Manhatten distance. Green: Euclidean Distance

As we discussed earlier, choosing a good heuristic function is key for the algorithm to function properly. In Figure 8 we see two examples of such functions. First, the orange line presents the Manhatten distance. It is defined as such:

$$Man(n) = |x_2 - x_1| + |y_2 - y_1|$$

This function does *not* fulfil our admissible requirement, in that it overestimates the optimal $h(n)$. The solution provided by A* would therefore not result in it being the shortest path.
Our second heuristic is the euclidean distance:

$$Euc(n) = \sqrt{|x_2 - x_1|^2 + |y_2 - y_1|^2}$$

This function does fulfil our admissible requirement. Furthermore, in our grid world, we know that this heuristic function is also consistent.

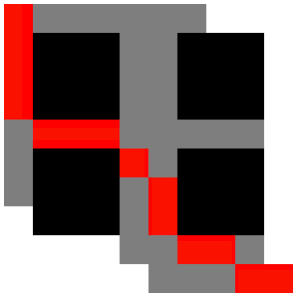

Figure 9: A* using euclidean distance - Cost 15.0711

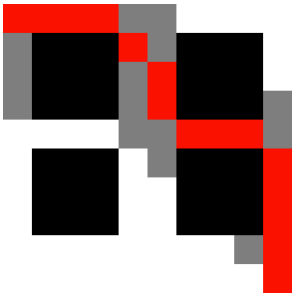

Figure 10: A* using manhatten distance - Cost 15.6569

If we look at Figure 10 using the Manhatten distance, we can see that we visit fewer nodes than the Euclidean distance in Figure 9. We also get an approximate path that reassembles that of the euclidean distance, but the path is no longer the shortest path.

Another thing to take into consideration when defining the heuristic function is the CPU cost. Calculating the Manhatten distance is supposedly cheaper, as this calculation is just summations. Finding the square root of something possibly is more computationally expensive, thereby increasing the cost of the Euclidean heuristic.

We could consequently sacrifice an optimal solution for decreased calculation time.

A* can provide some great improvements when used with a good heuristic for the given picture. However, when comparing the worst case, A* can perform no better than fx Dijkstra's algorithm, which ran at $O(n^2)$. Most of the time, however, A* will explore considerably fewer nodes than Dijkstra's algorithm, and thereby have a considerably improved runtime.

2.5 HPA* Shortest Path

The next algorithm of this thesis is the `Near Optimal Hierarchical Path-Finding` algorithm, also known as HPA*. This algorithm was made in 2004 by a group of scholars at the University of Alberta, Canada. (Botea, Müller, and Schaeffer 2004).

The algorithm operates by taking a problem domain and splitting it into smaller problem domains, thereby abstracting away a lot of useless data. This approach resembles the divide-and-conquer approach described in *Introduction to Algorithms*, in that it (Cormen et al. 2009):

- **Divides** the problem domain into smaller problem domains that are instances of the same problem
- **Conquers** the smaller problem domains. If the subdomains are too large, we divide further.
- **Combines** the solutions to the subproblems into a solution for our original problem.

The paper claims that HPA* may be up to 10 times faster than a highly-optimized A*, finding paths that are within 1% of optimal. (Botea, Müller, and Schaeffer 2004, p. 3)

The first step of HPA* is to preprocess the map. That is, we take a grid and divide it into a new set of rectangular areas called *clusters*. (ibid., p. 8). Once separated, we travel the border of each cluster pair c_1, c_2 (A cluster pair is two clusters that are adjacent to each other either horizontally or vertically) to identify a set of entrances.

An entrance is defined as being the length of a obstacle-free segment along the border of two adjacent clusters c_1 and c_2. This forms a line in each cluster l_1 and l_2. Each line consists of an amount of pixels p and for each $p \in l_1$ a symmetrical $symm(p) \in l_2$ exists.

As long as each entrance does not contain any obstacles, it is extended until it either reaches the end of the clusters or reaches an obstacle in either cluster.

When we have defined such an entrance e, we can define either one or two transitions. The authors propose the use of a predefined constant. In our case, we will use the constant 5. While this speeds up the algorithm, it gives up the optimal path in the initial problem, so the solution found is no longer certain to be the shortest path. (ibid., p. 12)

If a line l is larger than this constant, we make two transitions - one at the start and the end of the entrance. Otherwise, we make a single transition in the middle of l_1, l_2, that connects the two clusters c_1 and c_2.

We call such a transition an *edge* - and more specifically, an edge that connects two clusters are called an `INTER`-edge. Such an edge always have the length 1. (ibid., pp. 8–9). Figure 11 shows an example of these edges.

Moreover, for each pair of `INTER`-edges in a cluster c, we compute the path between these. For this purpose, we can choose any pathfinding algorithm, such as Dijkstra's Algorithm or A*. If such a path exists between the points, we

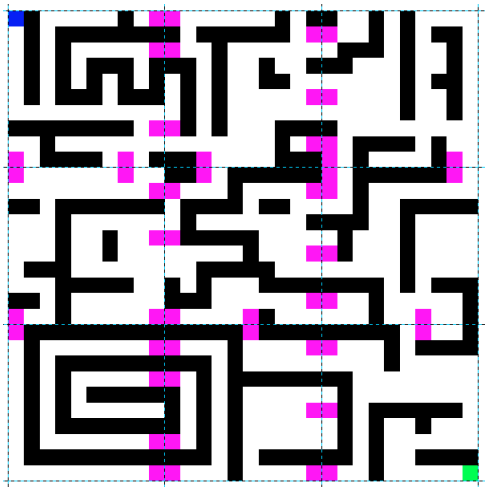

Figure 11: The inter-edges marked in pink for an example-maze. The blue-dotted line divides each cluster.

define an `INTRA`-edge. This path can either be stored for later use, at the cost of more memory or discarded at the cost of extra computational power. (Botea, Müller, and Schaeffer 2004, p. 10). In our instance, we will store the path for each `INTRA`-edge.

Lastly, we can insert our two pixels that define our start- and end goal. We then calculate the `INTRA`-edges for these, if any exists, between them and each transition point. The final result of such an abstract layer can be seen in Figure 12.

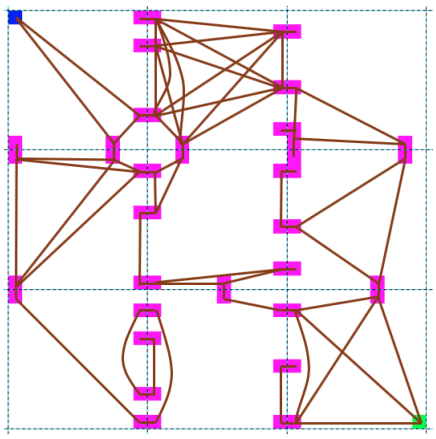

Figure 12: The first layer of the abstract graph

These edges make up our abstract graph. One may, if one wishes, continue to add abstraction layers. Adding another abstraction layer is yet again a question of finding entrances that live up to the criteria set forth as explained above. The benefits of a higher level of abstraction are increased speed, as there are

fewer edges to traverse. In this thesis, we will limit ourselves to only exploring the first level of abstraction.

In Figure 11 we see a total of 900 pixels (30x30). In Figure 12 we have reduced the 900 pixels to 27 `INTER`-edges and 50 `INTRA`-edges, for a total of 77 edges. That means we've removed 823 pixels that would have to be examined, and are left with only 8, 56% of the original problem.

This means that we will only have to use a path-finding algorithm on the 77 edges to find a solution. The real path can then be found by either recalculating the path between each edge or doing a lookup if the path was stored earlier.

As seen above, there can be a lot of benefits to using this method - nevertheless, it also has some downsides. Preprocessing a maze is very computationally expensive, as we have to calculate the path between each transition node, to make sure whether or not a valid `INTRA`-edge exist. The authors themselves also points this out, stating that:

"The efficiency of the method decreases when many obstacles are present and/or their shape is not a convex polygon." -Botea, Müller, and Schaeffer 2004, p. 6

`INTER`-edges are, on the other hand, not as computationally expensive. We will have to traverse a max of:

$$pixels = 2(\frac{img_h}{\text{clusterSize}} - 1) * (2 * img_h)$$

This equation is true, as long as the picture is of equal size both vertically and horizontally. The algorithm does work on pictures that is not of equal size, as long as a picture can be divided into equally sized clusters. In our case, we will limit ourselves to exploring pictures of equal size.

The first part of the calculation solves for, how many times you can draw a line that separates all clusters either horizontally or vertically (that is, how many l_1 and l_2 we have got in total). Secondly, we multiply by the number of pixels that is present in such a line ($l_1 + l_2$). An `INTRA`-edge may visit the same pixel several times, in that it has to calculate a path for each pair of transitions in a cluster.

The computation only has to be done once on a static map. If the map is dynamic, however, a single change in the map can be limited to recalculating the paths for the cluster, in which the change occurred. If the entire map changes, every cluster will have to be recalculated. This means that the optimality of the algorithm depends on the individual case. If we, for the same maze, want to test several different start- and goal nodes, we only have to recalculate the `INTRA`-edges in the cluster from which the two vertices are inserted. This leaves the rest of the abstract layers intact, decreasing all future requests for a given map.

The worst-case scenario for HPA* is when the border alternates between free tiles and obstacle tiles, while still letting all `INTER`-edges connect. That would mean that for a $n \cdot n$ cluster one would have to calculate a total of $n(2n - 1)$ paths. (ibid., p. 16) A good example is the cluster in the centre of the top row

clusters of Figure 12. This cluster has 15 `INTRA`-edges for only 6 `INTER`-edges, while the bottom centered cluster has 4 `INTRA`-edges to 8 `INTER`-edges.

2.6 Jump Point Search Shortest Path

The last searching algorithm that we are going to examine is the Jump Point Search algorithm, that was discovered by `Harabor & Grastien` in 2011. The algorithm is different from our other algorithms in that it only works on an undirected uniform-cost grid map, where horizontal- or vertical movement costs 1 and diagonal movement costs $\sqrt{2}$. (Harabor and Grastien 2011, p. 1115) This fits perfectly for the grid-based world that has been created so far.

The algorithm operates by 'skipping' a lot of the work done on a lot of pixels, by following a set of rules. The basic idea is that we only want to explore the pixels that we see as important and that may lead to other solutions. It does so by setting out a few simple rules.

First of, whenever we add a node to the queue, we add a direction $\overrightarrow{d}$ from which pixel it came from. This can be horizontally, vertically or diagonally. We then start by examining a node, by jumping horizontally, vertically and then diagonally in that order.

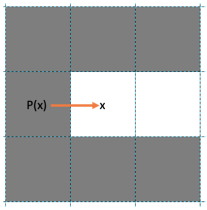

Figure 13: Moving horizontally or vertically. Every grey pixel can be visited more efficiently through our parent p(x) pixel.

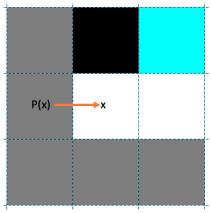

Figure 14: Moving horizontally or vertically. The blue pixel is our forced neighbour. The black is an obstacle.

Figure 13 is an example of us moving horizontally, with no obstacles. Every other node than the one in the direction that we moved from $P(x)$ to x can be reached more efficiently or equally by taking another route, that does not go through x. The assumption for this rule goes, that the pixels do not need to be evaluated, as they would provide an equal- or longer path, to the one we are currently taking. This leaves only one pixel left to be examined, which is the white pixel in the direction that is being stepped.

Obstacles change this beheaviour. As seen in Figure 14, if an obstacle is directly above x, it can no longer reach the blue node through other paths that do not involve x, that would be shorter- or equal to paths that do involve x. (ibid., pp. 1115–1116)
We make similar rules when moving diagonally. While no obstacles are present, all grey nodes in Figure 15 can be reached more efficiently through a path not

involving x. Diagonal moves are different, in that it does not disregard paths that are equal to the ones involving x.

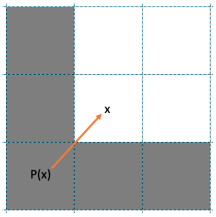

Figure 15: Moving diagonally. Every grey pixel can be visited more efficiently through our parent p(x) pixel.

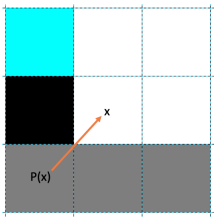

Figure 16: Moving diagonally. The blue pixel is our forced neighbour. The black is an obstacle.

Finally, we make two assumptions. If a horizontal or vertical move meets an obstacle, it returns null. We assume that, since every other grey node is equivalent in the path, the path without the obstacle will be examined at some other point. An example is shown in Figure 17. (Harabor and Grastien 2011, p. 1116)

We can not move diagonally to a forced neighbour or a pruned neighbour, if two pixels together create a wall, as shown in Figure 18. This is not a move limited by the paper itself, but rather a move that is not allowed due to the nature of such a move for our grid-based world.

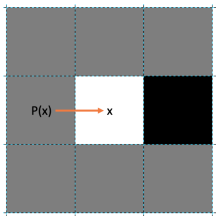

Figure 17: Moving horizontally, we terminate, due to meeting a wall. The grey pixels are assumed to be explored at some other point.

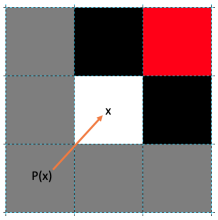

Figure 18: Moving diagonally, the red pixel is unreachable due to the obstacle directly above- and to the right of x.

Combining these rules, one can now create an algorithm, that when followed, will return the shortest path.

When run, the algorithm takes the following steps: First of, we run a queue that prioritises the next pixel to be examined just like A*. That is, we use the current weight plus a heuristic to determine the next in the queue. We only add a pixel to the queue if either it is unvisited or that the new weight is lower than the one we have at the moment. Once a pixel has been examined, we never

examine it again.

Our start pixel s is added to the queue with all neighbours that are not obstacles our outside our grid. We start by moving vertically- or horizontally. If we meet a pixel that has a forced neighbour, we add the pixel to the queue. Once the horizontal- and vertical move has finished, we move a step in the diagonal direction, if any. At the diagonal pixel, we then explore horizontally and vertically. If there are any forced neighbours, we add the diagonal pixel itself to the queue. If we find our goal, we add our goal to the queue.

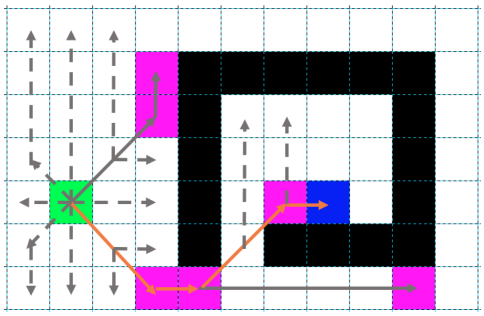

Figure 19: An example run of JPS. The orange arrows are the shortest path. The purple pixels are jump points found. The dotted arrows are recursion without jump points while the bold arrows are paths to jump points

We can see an example of a run of JPS in Figure 19.

As a final important note, `Harabor & Grastien` notes that it is essential that before each diagonal step the algorithm must first fail to detect any straight jump points. This condition is vital for preserving optimality. (Harabor and Grastien 2011, p. 1116)

This algorithm can be very efficient in obstacle-full environments. The problems it may face is that in obstacle-free environments, it can quickly have to visit each pixel, even though no work is done on these. Here, A* might perform better than JPS. But JPS strength lies in its rules, so an obstacle-full environment should show favourable for this algorithm. The algorithm does not require any preprocessing either, compared to HPA*.

Our worst-case running time is therefore not improved over Dijkstra's $O(n^2)$, but our amortised running time should be considerably less.

2.7 Other notable algorithms

This thesis only describes a few of many other notable algorithms. A few of the other solutions to the shortest path problem are introduced here but are not explained in depth.

Our world was described as having only positive weights - i.e. a weight of 1 on the horizontal- and vertical movements, and $\sqrt{2}$ on the diagonal movements. This means that we never need to examine a negative weight. Since our world is digital, we could have easily made weights negative in some cases and this would have broken algorithms such as BFS and Dijkstra. For this, one

could have used the **Bellman-Ford** algorithm. This allows negative weights, as long as there are no negative cycles.

The Bellman-Ford algorithm is the work of two independent mathematicians, `Richard Bellmann`, who published a paper in 1958 (Bellman 1958) and `L. R. Ford`, whom described the general framework in his book. (Ford and Fulkerson 1962). It is hard to find when exactly the two papers were combined into one, and people started referring to the algorithm as Bellman-Ford. We do not examine this algorithm, as the negative weights limit the amount of comparable testing and we would have to disregard algorithms on tests that may have included negative weights.

Of course, with the rise of computer games, the question of finding a better pathfinding algorithm has been discussed. As such, a lot of solutions have been tested and tried within just the last decade. One such algorithm is **Theta***, which was published in 2007 and again later described in their 2010 paper. (Kenny et al. 2014-01-16). This is an algorithm that does not guarantee a shortest path solution but trades this to achieve a shorter runtime. As is seen with a lot of the newer algorithms, they expand on the properties of A* just like HPA*. This algorithm is also described on a grid, and as such would function on our world. Instead of Theta*, HPA* was chosen, as HPA* changed the data-structure and required preprocessing.

This is only scratching the surface of a deep ocean of knowledge. A lot of other algorithms present their solutions to the single-source shortest path problem as well as all-pair shortest path problem.

3 Implementation

3.1 Tools Used

To run the program, a list of tools was used. The program itself was chosen to be programmed in C++. The programs used to compile are as follows:

- **CMake V. 3.15.3** - An open-source set of tools to allow for cross-platform build, test and design of software.
- **GCC 9.2.0** - The compiler that converts C++ into executable code
- **(Mac only) XCode 11.0** - Apples set of tools used for development on macOS.
- **(Mac only) XQuartz 2.7.11** - The open-source set of tools that allows for different kind of windows to be built on macOS.
- **ImageMagick 7.0.8-66** - A free and open-source picture editing software. This was used to deal with the compression of images, such that the data was freely available, instead of having to deal with uncompressing the picture.
- **CImg 2.7.2** - An open-source C++ library that allows one to load a picture, retrieve colour values and edit properties of the picture as well as displaying the picture on the screen.

All the code is freely available to be examined in the GitHub repository at `https://github.com/tikki100/SPBachelor/`. (*Shortest Path Bachelor Code Github* 2020).

3.2 Creating the Tests

In the next section, I will mention colours by their values $\{0-255, 0-255, 0-255\}$ where each value represents the intensity of the colour $\{\text{Red}, \text{Green}, \text{Blue}\}$.

The program, once initialised, takes a `png` picture and loads it into memory. From here, it first checks that the following conditions are true:

1. There are only 3 colours on our spectrum - Red, green and blue. This is to prevent alpha colours from being present.
2. There is one and only one starting pixel with the colour $\{0, 255, 0\}$
3. There is one and only one goal pixel with the colour $\{0, 0, 255\}$.

If these conditions are met, the program may continue.

For our implementation, we have limited the test to mazes that are equal in both height and width. This is to ease the process of HPA* which divides the picture into equally sized clusters.

Each algorithm keeps track of a table of `cost_so_far` as well as `came_from`. These tables are of the type `std::unordered_map`, which means that a lookup or assignment, given a proper good hash function for the key-value, should

take amortised of O(1). The only cost should then be the increased memory, being 24 bytes for the map itself and 8 bytes for each set of pixel coordinates. Breadth-first search does also keep track of a $cost_so_far$ table, even though it does not use this in the algorithm. This is such that the final path can be compared in terms of the final weighted cost, such that we can see if an optimal or suboptimal path has been found.

Each tests contains 5 sizes of the respective test - 100x100, 200x200, 300x300, 400x400, 500x500, with the exemption of one test, where the maze generator was limited to 400x400. A picture was then generated for each size. The reason as to why we generate a new picture for each size is due to the layout of the maze. If one was to generates a maze of the size 400x400, all paths are no longer connected, when cropping the picture.

We ensure that the picture only contains black pixels with the value $\{255, 255, 255\}$ or white pixels with the value $\{0, 0, 0\}$. This is because, when we test a pixel for whether or not it is walkable, we have a whitelist of only 4 values. These are white: $\{255, 255, 255\}$, grey: $\{126, 126, 126\}$, purple: $\{255, 0, 255\}$ and our goal node blue: $\{0, 0, 255\}$. All other nodes are marked as obstacles. Purple and grey nodes are used for debugging purposes and do not appear in the final result.

For the tests, the following pictures were created:

- **Test1:** An obstacle-free picture
- **Test2:** A squared maze (*Diagonal Maze Generator* 2019)
- **Test3:** A hexagonal maze. (*Hexagonal Maze Generator* 2019)
- **Test4:** A picture with random pixels
- **Test5:** A checkered picture

Test 1 was generated using an image editor.

Test 2 was generated using the maze generator. (*Diagonal Maze Generator* 2019). Due to the limitations of the generator, the maximum size available was 400x400.

Test 3 was generated using another maze generator (*Hexagonal Maze Generator* 2019) and afterwards using an image editor to increase the contrast to ensure that only black/white pixels remained.

Test 4 was generated by taking a number at random between 1 and 100. If the number was equal to or below 10, the pixel would be coloured black. This generated the random pixels across the picture.

Test 5 used a similar script to create $7x7$ squares with a distance of 3 between each other, instead testing if a picture was within a predetermined limit.

3.3 Measuring the algorithms

To compare the different algorithms, we compare running time, memory usage and accuracy of the algorithms.

While historically, a lot of authors did not concern themselves with the running time of their algorithms, i.e. Dijkstra makes no mention of the running

time, but rather if his algorithm works or not, (Dijkstra 1959) we will use this measurement of time. Although, it is important to discern that he did concern himself in how the algorithm did against other algorithms such as the one introduced by `Ford`.

We use this measurement as, intuitively, it makes sense to compare two algorithms run time against each other, when they need to calculate the same solution (i.e. the shortest path) for the same problem. An algorithm that always finds the correct result, but takes hours to complete is much less preferred than an algorithm that takes minutes to complete, yet still always finds the correct result.

Furthermore, we make use of a second measurement, which is memory usage. It is important to record this measurement as well, as an algorithm often can 'trade' between storing information in memory for reduced computational time and vice-versa.

Thirdly, we record the final path length of the algorithm. This makes it possible for us to compare whether a shortest path has been found, or an alternative route has been taken. Breadth-first search and HPA* may find solutions that are not the shortest path, so measuring this will allow us to find out, how much each algorithm deviates from the optimal shortest-path solution.

4 Results

While this section presents the results as graphs, all results may also be found in their raw form in the appendix B.

The program from which the tests were executed was run on a Macbook Pro (13-inch mid-2012) with an Intel Core i7 (I7-3520M), 2.9 GHz processor, and 16 Gb 1600 MHz DDr3 Ram. Complete computer specs can be seen by following the link, (*Apple MacBook Pro "Core i7" 2.9 13" Mid-2012 Specs* 2019), whereas the RAM has been replaced with the above and the optical drive has been replaced for a Samsung 850 SSD.

Furthermore, the Macbook had a charger connected to avoid the OS slowing the tests down because of battery issues.

The program was compiled with the "-O3" flag to get the most optimisations out of the compiler for both runtime and memory optimization.

HPA* has been run on all tests with a constant cluster size of 10 for all tests. Preprocessing will be shown as an independent result for all tests.

The runtime was calculated by exploiting the `C++` convention, that when an object is created within a set of brackets, it is destroyed when leaving this set of brackets. For each algorithm, a timer class was created before running the algorithm. The timer class saved down the current time. Once the algorithm had finished and left the brackets, the object was destroyed, saving and printing the difference between the time of destruction and creation. All tests were run within a for loop, looping over all 24 tests with all 5 algorithms for the runtime results.

The memory was calculated differently. To avoid memory leaks from affecting results across different tests and algorithms, each test was run independently of one another. This was done in `XCode`. When the program was executed, it had been modified slightly to take the input of the user, such that one would enter the number of the test to run for the current algorithm. A breakpoint was then placed at the end. When the breakpoint was hit, the highest amount of memory used was recorded. This was done for each of the 24 tests and 5 algorithms.

Deviations in percent are calculated as such:

$$\frac{deviation - shortestpath}{shortestpath}$$

4.1 Test 1

The first test was a blank picture, where the algorithms had to find their shortest path from the top left to the bottom right. An example is shown in Figure 20.

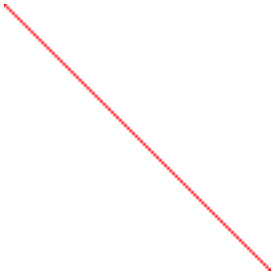

Figure 20: An example solution to test 1 of size 100x100

The test was created in 5 sizes, from 100x100 to 500x500.

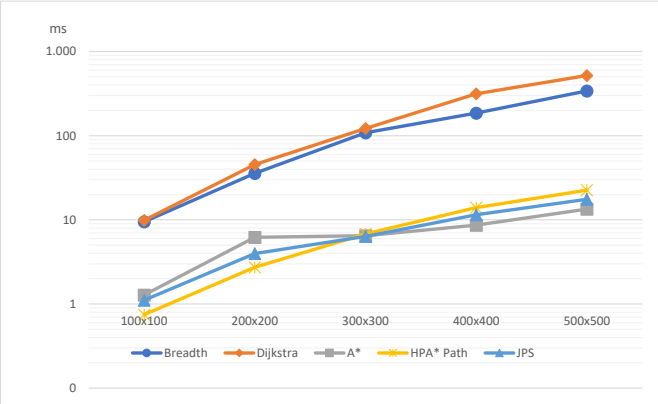

Figure 21: Runtime results for test 1

The results, as seen in Figure 21 for test 1, shows that the algorithms with the worst runtime were Dijkstra's in all cases and the worst-case runtime of 519,384 ms.

In the smaller tests, HPA* is the fastest, clocking in at 0,747 ms compared to Dijkstra at 9,878 ms while in the biggest picture, A* is the fastest at 13,420 ms.

The largest difference is in the 500x500 picture, between A* and Dijkstra, with a difference of 505,964 ms (3.870% times slower).

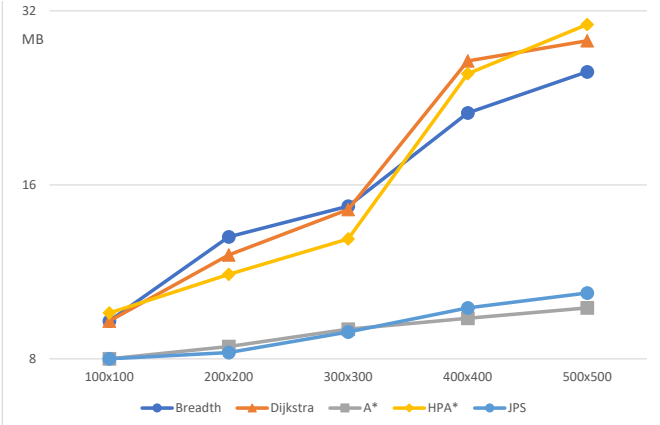

Figure 22: Memory results for test 1

In this case, we see that Breadth, Dijkstra and HPA* follows each other in memory usage maxing out at 30,3 MB in Figure 22. In the later stages, breadth is a bit less, using up to 25,1 MB. Jump point search and A* uses significantly less, lying at max 10,4 MB.

The only algorithm with deviations was HPA*.

	100x100	**200x200**	**300x30**	**400x400**	**500x500**
Shortest Path	140.007	281.428	422.85	564.271	705.693
HPA* Diff	+5.272	+11.13	+16.988	+22.846	+28.703
HPA* % Diff	3.77%	3.96%	4.02%	4.05%	4.07%

4.2 Test 2

The second test was generated using an online maze generator. (*Diagonal Maze Generator* 2019). A different maze was generated for each size, to ensure that the algorithm had to find its path from the top left to the bottom right. An example of the maze can be seen in Figure 23.

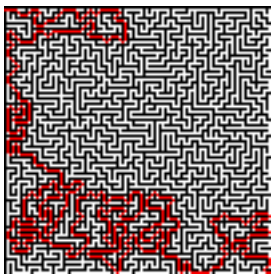

Figure 23: An example solution to test 2 of size 100x100

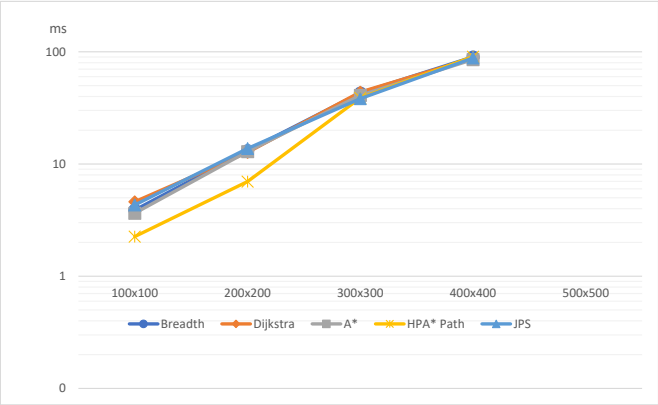

Figure 24: Runtime results for test 2

In Figure 24 we see that all algorithms except HPA* follow each other in terms of runtime, until later, where all algorithms are more or less equal in runtime. Early on, the best case HPA* takes 2,257 ms and the worst-case Dijkstra takes 4,613 ms. HPA* is therefore double as fast in this specific case.

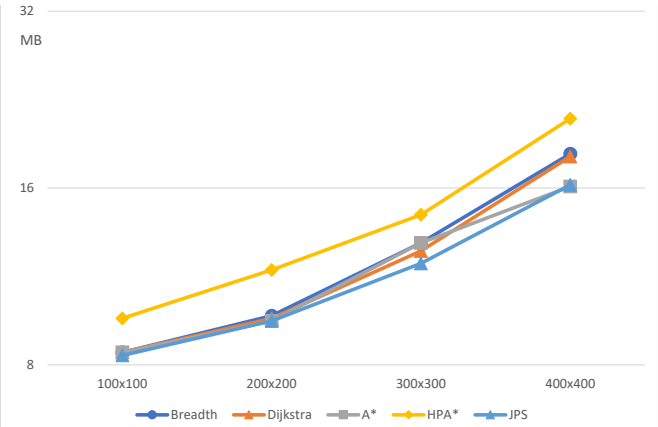

Figure 25: Memory results for test 2

In terms of memory usage, we see in Figure 25 the algorithm that uses the most memory is HPA*. The remaining algorithms are close to each other in terms of average memory usage, with Jump Point Search using the smallest amount of memory at all times.

The only algorithm with deviations was HPA*.

	100x100	**200x200**	**300x300**	**400x400**
Shortest Path	859.205	2214.28	4198.55	8376.09
HPA* Difference	+35.733	+87.87	+179.84	+362.01
HPA* % Difference	4.16%	3.97%	4.28%	4.32%

4.3 Test 3

Test 3 was once again created using an online maze generator. (*Hexagonal Maze Generator* 2019) This maze is different from the previous maze, in that the shortest path exists from the top centre to the bottom centre. This test was generated in 5 different sizes, ranging from 100x100 to 500x500. An example can be seen in Figure 26

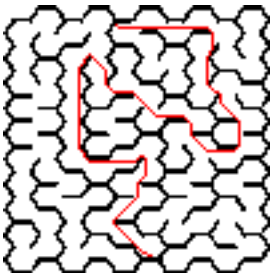

Figure 26: An example solution to test 3 of size 100x100

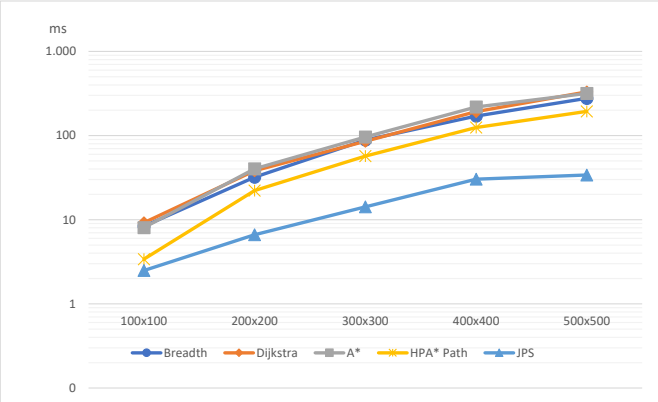

Figure 27: Runtime results for test 3

In Figure 27 we see that the 3 algorithms that take the longest time are Dijkstra's, A* and breadth-first search, with Dijkstra clocking in at its worst case at 330,626 ms. HPA* is slightly shorter in runtime at 193,844 ms, while JPS* is up to 10 times shorter than Dijkstra, clocking in at 33,995 ms.

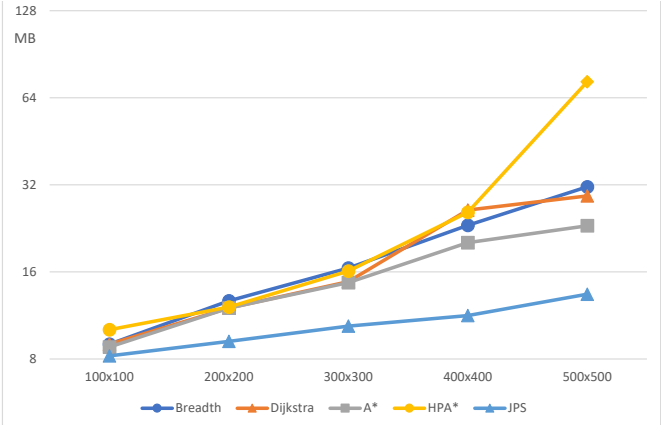

Figure 28: Memory results for test 3

In Figure 28 we begin to see the first outliers in memory usage. HPA* uses up to 72,7 MB memory at the 500x500 test, while JPS only uses 13,4 MB. Breadth-first search and Dijkstra are quite equal in memory usage, but still over half than that of HPA*.

The only algorithm with deviations was HPA*.

	100x100	**200x200**	**300x300**	**400x400**	**500x500**
Shortest Path	276.137	650.558	819.423	1890.88	1223.38
HPA* Diff	+25.172	+30.929	+52.628	+102.450	+69.48
HPA* % Diff	9.12%	4.75%	6.42%	5.42%	5.68%

4.4 Test 4

This test was created by telling an algorithm to choose the colour for the pixel. The pixel had 10% chance to black and 90% to be white in 5 different sizes, from 100x100 to 500x500. An example can be seen in Figure 29.

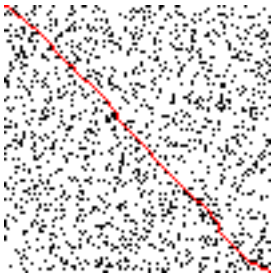

Figure 29: An example solution to test 4 of size 100x100

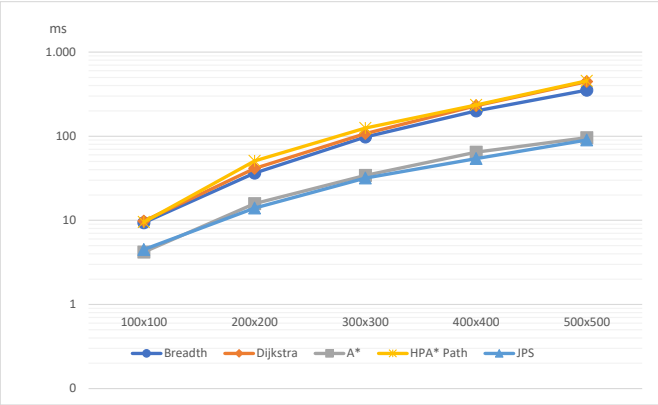

Figure 30: Runtime results for test 4

The algorithm that took the longest, in this case, was HPA* with a time of 451,789 - while Dijkstra took 435,859 ms. That is, a difference of 15,93 ms in the worst case, as seen in Figure 30

There is a fight between A* and JPS in being the fastest algorithm, with the 100x100 clocking JPS as the winner by 0,301 ms as well as at 500x500, by 6,087 ms.

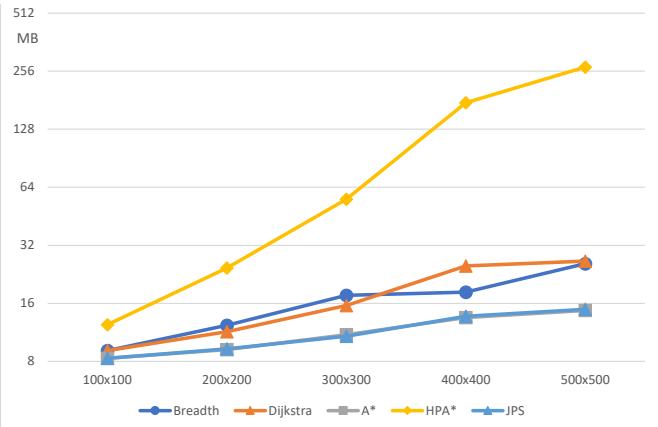

Figure 31: Memory results for test 4

We can see that in Figure 31 HPA* uses up to 268,9 MB for the 500x500 for this problem, while JPS only 14,7 MB. Throughout this entire problem, HPA* uses more memory than all other algorithms.

This test showed deviations in two algorithms. HPA* and Breadth both had deviations. This is both to be expected, as neither is guaranteed to return the shortest path.

	100x100	**200x200**	**300x300**	**400x400**	**500x500**
Shortest Path	145.865	290.215	435.737	583.016	727.367
Breadth Diff	+0.828			+0.829	
Breadth % Diff	0.57%			0.14%	
HPA* Diff	+6.444	+14.059	+21.674	+27.532	+39.833
HPA* % Diff	4.42%	4.84%	4.97%	4.72%	5.77%

4.5 Test 5

The last test was created by using an algorithm to create black squares of 7x7 with 3x3 columns and rows of white. An example is shown in Figure 32.

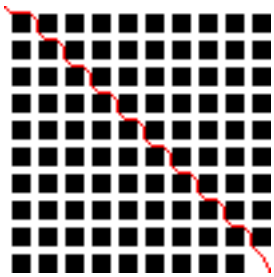

Figure 32: An example solution to test 5 of size 100x100

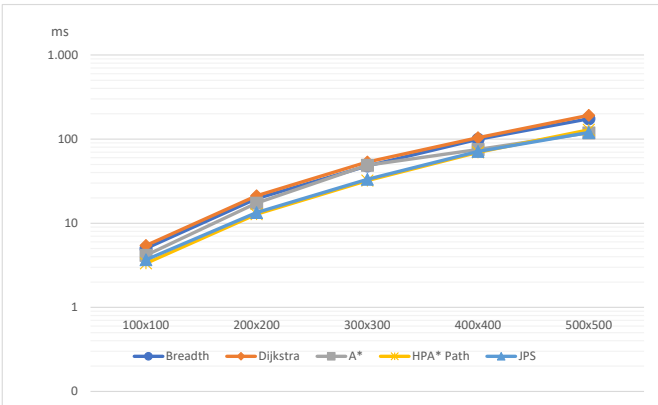

Figure 33: Runtime results for test 5

The algorithms are quite equal in this. JPS and A* are the fastest, at 119,793 ms and 118,748 ms while Dijkstra and Breadth are the slowest at 191,730 ms and 174,250 ms as illustrated in Figure 33.

HPA* follows quite nicely along in the middle of A* and HPA*, sometimes faster, sometimes slower by a couple of ms.

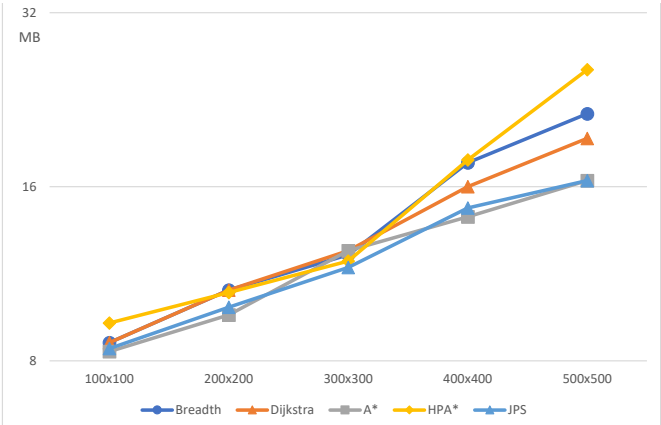

Figure 34: Memory results for test 5

In memory usage, we see a spike in the end for HPA* in Figure 34, using 25,5 MB. Breadth used 21,4 MB and Dijkstra used 19,4 MB in the worst case.

Early on, all algorithms follow each other closely in memory usage. A* and JPS follow each other closely throughout the entire picture sizes.

There was only one algorithm with deviations in this case: HPA*.

	100x100	**200x200**	**300x300**	**400x400**	**500x500**
Shortest Path	162.853	327.706	492.558	657.411	822.264
HPA* Diff	+19.917	+43.348	+66.78	+90.211	+113.643
HPA* % Diff	12.23%	13.23%	13.56%	13.72%	13.82%

4.6 HPA* Preprocessing

This section illustrates the preprocessing time that HPA* had to take for each test. This result was recorded seperately from pathfinding, as to compare the gains from a preprocessing algorithm such as HPA* to other competitors.

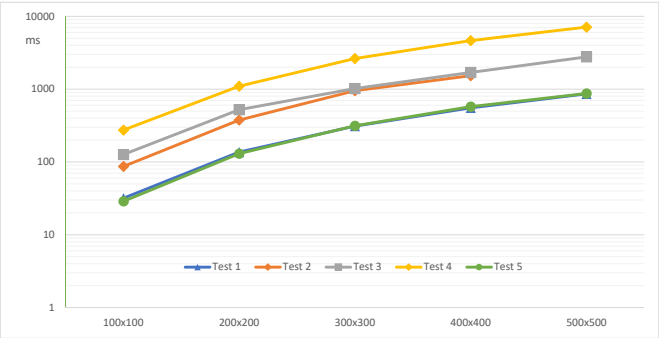

Figure 35: HPA* Preprocessing in ms for all tests

In Figure 35 we see that Test 4 with the random pixels was the test that took the longest time for HPA* to preprocess, clocking in at a worst-case running time of 7064,12 ms, while test 5 was almost just as fast as the empty picture in test 1 in terms of preprocessing.

5 Discussion

5.1 In General

In this section, we will examine the results, what has been learned from these and how it can be used to further improve future pathfinding.

C++ can have many pitfalls if one does not pay attention. Memory leaks are common and runtimes may be significantly affected if one does not take into account to optimize one's code. One such example was the use of `std::map`, which kept track of the weights of each pixel as well as each pixels parent. `std::map` has a complexity of $O(\log size)$, which can increase runtimes by a considerable amount. C++ provides an alternative, which is `std::unordered_map` that is amortized constant and worst-case linear of the size, which is a significant increase in speed. Pitfalls as these can skew the results, as an algorithm which maps a lot of this data may be seen as significantly slower, even though this might not be the case. One could argue for whether this is simply a part of the usage of the algorithms in the specific environment.

5.2 Breadth-first, Dijkstra and A*

Test 1 was designed in such a way that it would challenge Dijkstra and Breadth-first in ways, which we knew that the worst-case runtimes would be challenged. Here, they had to examine the entire picture, before being able to find a solution. An interesting observation that was made is that Dijkstra is slower than breadth-first, although not significantly. This could be because of the aforementioned reasons. A lookup in the `std::unordered_map` table could

slow down the algorithm by an amount big enough to be present in data. If we examine the other results, we see that they are are always close to each other, although in some cases, breadth may be a bit faster than Dijkstra - e.g. in test 1. In our environment, Dijkstra acts almost as a breadth-first search, but it prioritises examining horizontal- and vertical neighbours first, before examining diagonal neighbours. The memory usage is also close to each other.

A surprising fact is that even though breadth-first is not accurate in terms of the shortest path, there were only one out of five tests where breadth-first did not find the shortest path - in test 4. We see a deviation between the optimal path and breadth-first path of only 0.57% in this case. But when one comes to think of it, the results are not as surprising, as Dijkstra is also a kind of Breadth First Search. The deviations should, therefore, be minimal as the results also prove.

The benefits of using an algorithm such as A* is obvious in test 1 and test 3, where it largely outperforms a lot of other solutions. Test 1 shows the best-case scenario, where no obstacles are present and it is displayed in both runtime and memory usage. When we introduce more complex mazes with more obstacles, such as in test 2 and 3, we see how A* may perform badly, increasing in both memory usage and runtime. A* does in some cases use less memory than Dijkstra's Algorithm and Breadth-First searches, showing the improvements that may be gained, as is the case in test 3.

Once again, the apparent benefits that A* may provide are displayed again in test 4 which results are close that of test 1. This shows that A* is not confused by the random data. The runtime is significantly lowered compared to that of other algorithms, being on pair with Jump Point Search.

5.3 HPA*

The HPA* algorithm was an engaging project to implement and proved to provide some rather deviating results compared to some of the other algorithms.

The weakness of this algorithm lies in it having to preprocess the problem. In nearly all cases, it was much faster to run any of the other algorithms, if one was interested in finding the shortest path. The strength in this algorithm lies in speeding up all future pathfinding requests. This is also why the pathfinding is shown independently of the preprocessing. When examining the code, it was found that the runtime was increased, when the `INTRA`-edges had to be created. This means that the preprocessing step spends a lot of its resources there.

Another weakness, according to my personal opinion, is that one has to do tuning of the cluster size variable. (Botea, Müller, and Schaeffer 2004, p. 19) This provides another layer of work, whether this is done manually or is automated. This is why a static constant was chosen throughout the tests - both to see the strengths and weakness given a good or bad cluster size parameter. One could compare it to choosing a heuristic for the other pathfinding algorithms - a choice has to be made by the user of the algorithm, that affects the quality of the solution.

A picture such as test 2 shows how HPA* works efficiently in mazes. I believe, that if one were to tune the cluster size variable, the result may improve on the bigger sizes, but as could be seen, HPA* proved faster than all other

algorithms in the smaller sizes. Test 2 didn't have a big preprocessing time either, indicating the creation of few `INTRA`-edges. The same is true for test 5. In general, if the preprocessing time is low, the abstract pathfinding is also fast, as the number of abstract edges is reduced.

Later on, we see how HPA* may be weak. In test 4, which resembles that of a worst-case scenario for this algorithm (Botea, Müller, and Schaeffer 2004, p. 16), as there was a lot of `INTER`-edges that were connected to each other. We can see the increased work on the preprocessing size and especially on memory usage. Test 4 saw memory usage *up to* $268,9$ *MB!* An insane amount, considering that of all other algorithms, the highest amount of memory used across all tests were $31,5$ MB by breadth-first search in test 3. It is to be considered that this may be a side effect of inefficient programming, but we assume that the results are to be trusted in this case. C++ is a programming language, where memory may go rouge if one does not pay attention. If one is looping over an array of values, one may end up copying each value instead of working directly on the values in the array, and this memory may never be freed. The fault may very well lie in such an error.

Of course, the algorithm suggests that one may trade the memory in addition of the extra computational time to refind the path when moving from the abstract layer to the normal layer. Considering the computational time is already high for most cases, it is highly dubious whether or not this trade would be worth it.

I couldn't help but think of ways that one may improve the runtime of this algorithm. Since we are working on mazes in general, we want to avoid having to search through mazes without a solution to our problem. Since the CPU-resources was spent creating the `INTRA`-edges, one could perhaps speed up the solution by performing an abstract search on all the `INTER`-edges. This would mark possible clusters that could contain a solution. `INTRA`-edges could then be calculated for the said clusters, and the search could repeat itself with the now inserted `INTRA`-edges.

This may end up improving the preprocessing time, but it changes fundamentally the benefits and cons of the algorithm, as this could wind up being slower than the algorithm in its original state. We saw, that at times such as in test 1, HPA* search can perform just as well as JPS and A*.

Another solution would be to change the pathfinding algorithm between the `INTRA`-edges. Instead of using A*, one may instead implement Jump Point Search.

Finally, HPA* was one of the algorithms that did not provide an optimal solution. This tradeoff, compared to the high processing times and high memory requirements, showed deviations from 3.77% to 13.82%. This result may be due to the cluster size variable not being tuned correctly as it is not within the 1% shortest path as described in the original paper.

5.4 Jump Point Search

In terms of a general "winner", jump point search is a clear favourite. It is strong in computational time across almost all tests, with the exception of test 2, where the algorithm can not benefit from not having to explore a lot of otherwise clear

nodes. Even in test 1, which is one of the worst-case scenarios for the algorithm, the algorithm performs almost just as well as A*, which is running a best-case scenario.

Jump point search is also efficient memory-wise. We only record the paths to and from jump points, disregarding information that is deemed unusable (jumps that do not return jump points). An example of recording information is done when a jump point is found. The algorithm then backtracks to the current pixel being examined, saving the new weight and parent for each pixel on its way back until it meets the jump point being examined.

While the algorithm may seem simple on the surface, it was hard to implement, as the paper itself leaves a lot of the implementation details to the user. An example is the pruned neighbours function as well as the definition of forced neighbours. One has to pay careful attention to detail to get every orientation right.

There is also the question of what happens if a jump point is found more than once - and if at the second time, the jump point can be reached more efficiently. One has to keep a track of which jump points have been popped from the list, to avoid reiterating over the same jump point more than once, as well as being able to update the current weight of a jump point in the queue. A case I struggled with can be seen in Figure 36. Here, JPS would take a sub-optimal route. This error was caused because JPS at the time did not correctly re-examine pixels it had already visited, correctly calculating the new weight.

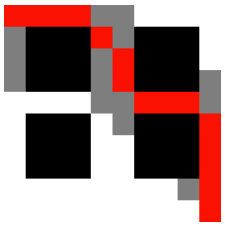

Figure 36: JPS would take the path marked in red instead of the shortest path.

A lot of these issues was found when testing test 4, as this contained a lot of challenges and specific cases, where JPS ended up taking an inefficient route if the logic of the program was wrong. This helped fix a lot of bugs in the early implementations, but it is definitively the algorithm that provided the biggest headache in terms of implementation.

The results show that the time spent implementing and understanding the algorithm was well spent, due to the increased running time. JPS is almost always the fastest in all cases and requires no tuning of the algorithm to make it work on different cases. The memory usage is also low, throughout all stages. Even when met with the worst-case scenario in test 1, the memory usage is only slightly higher than A* in the bigger pictures but lower- or equal to in the smaller pictures.

A weakness is that the algorithm only works on uniform-cost grids, but in most cases, one can make convert one version of a problem into another.

6 Conclusion

In this thesis, we wanted to examine the literature of shortest paths. We have done so, finding a lot on the history of the issue, stemming back to references from 1736. We selected five algorithms out of many more, to implement and test. Two algorithms did not provide an optimal shortest path, but an approximation: HPA* and Breadth-First search. The three remaining algorithms provided an optimal shortest path: Dijkstra's Algorithm, A* Shortest Path and Jump Point Search.

We learned that HPA* can be efficient in some cases, proposing that efficiency may be dependant on the cluster size variable being tuned for a specific problem. This tuning does require a lot of work and whether this process may be automated is not known. Any changes to map size will require a new fine-tuning of the cluster size variable. In comparison, other algorithms can match the optimal running time of HPA* when finding a path, without preprocessing the map.

In terms of grids, we learned that Jump Point Search is definitively a good candidate for pathfinding. Often, game worlds are made up of grids and as such, jump point search can be implemented. In small, narrow mazes, Jump Point Search loses some of its efficiency, but in almost all other cases, it remains the fastest and best option to run.

We have presented the results of implementing the different algorithms in C++, compared and discussed the results of each algorithm.

Future work on this thesis could be to examine the fine-tuning of HPA* further. That is, tuning the cluster size variable as well as testing different algorithms for creating the `INTRA`-edges of the algorithm. More future work could be trying to take abstract game maps and translate them into 2D grids. Often, environments such as 3D worlds has ground with a certain height. This ground exists in the 2D world. Thereby, we can simply draw lines, to make a grid. The consequences and benefits of such decisions could then be measured and compared, to see whether algorithms such as Jump Point Search would remain the most feasible candidate to use for pathfinding.

A Bibliography

References

Apple MacBook Pro "Core i7" 2.9 13" Mid-2012 Specs (2019). URL: https://everymac.com/systems/apple/macbook_pro/specs/macbook-pro-core-i7-2.9-13-mid-2012-unibody-usb3-specs.html (visited on Sept. 23, 2019).

Bellman, Richard (1958). "ON A ROUTING PROBLEM". In: Vol. 16.1, pp. 87–90. ISSN: 0033569X. URL: https://rex.kb.dk/permalink/f/i2hf7d/TN_jstor_archive_3343634538 (visited on Sept. 8, 2019).

Biggs, Norman L. (1986, cop. 1976). *Graph theory, 1736-1936*. Clarendon. ISBN: 0198539169. URL: https://books.google.dk/books?id=XqYTk0sXmpoC&lpg=PP1&hl=da&pg=PP1#v=onepage&q&f=false.

Botea, Adi, Martin Müller, and Jonathan Schaeffer (2004). "Near Optimal Hierarchical Path-Finding (HPA*)". In: Vol. 1. DOI: 10.1.1.479.4675. URL: http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.479.4675&rep=rep1&type=pdf (visited on Sept. 5, 2019).

Cormen, Thomas H. et al. (2009). *Introduction to Algorithms*. 3rd. The MIT Press. ISBN: 978-0-262-53305-8.

Diagonal Maze Generator (2019). URL: https://keesiemeijer.github.io/maze-generator/#generate (visited on Dec. 18, 2019).

Dijkstra, E. W. (1959). "A note on two problems in connexion with graphs". In: *Numerische Mathematik* 1.1, pp. 269–271. ISSN: 0945-3245. DOI: 10.1007/BF01386390. URL: https://doi.org/10.1007/BF01386390 (visited on Sept. 5, 2019).

Ford Lester Randolph, Jr. and D. R. Fulkerson (1962). *Flows in Networks*. eng. Princeton: Princeton University Press. ISBN: 9780691079622.

Harabor, Daniel and Alban Grastien (2011). "Online Graph Pruning for Pathfinding on Grid Maps". In: AAAI Conference on Artificial Intelligence, pp. 1114–1119. URL: https://www.aaai.org/ocs/index.php/AAAI/paper/download/3761/4007 (visited on Sept. 5, 2019).

Hart, Peter E, Nils J Nilsson, and Bertram Raphael (1968). "A Formal Basis for the Heuristic Determination of Minimum Cost Paths". In: *IEEE Transactions on Systems Science and Cybernetics* Vol. 4.2, pp. 100–107. ISSN: 0536-1567. URL: https://rex.kb.dk/permalink/f/i2hf7d/TN_ieee_s4082128 (visited on Sept. 8, 2019).

Hexagonal Maze Generator (2019). URL: http://www.mazegenerator.net (visited on Dec. 18, 2019).

Kenny, Daniel et al. (2014-01-16). "Theta*: Any-Angle Path Planning on Grids". eng. In: *arXiv.org* 39, pp. 533–579. ISSN: 10769757. URL: http://search.proquest.com/docview/2083886228/.

Koenig, Dënes (1990). *Theory of Finite and Infinite Graphs*. Cambridge, MA, USA: Birkhauser Boston Inc. ISBN: 0-8176-3389-8.

Moore, Edward F. (1957). "The Shortest Path Through A Maze". In: *Proceedings of an International Conference on the theory of Switching, Part II*. Mass., 1959.

Shortest Path Bachelor Code Github (2020). URL: https://github.com/tikki100/SPBachelor (visited on Jan. 12, 2020).

Tarry, G. (1895). *The Labyrinth Problem*. URL: https://www.cantab.net/users/michael.behrend/repubs/maze_maths/pages/tarry_lab_en.html (visited on Nov. 27, 2019).

Thorup, Mikkel (2000). "On RAM Priority Queues". In: *SIAM Journal on Computing* 30.1, pp. 86–109. ISSN: 0097-5397.

B Results

B.1 Runtime

These results were gained by running the program and using a class' creation and destruction as markers of when the algorithm starts and ends.

For this purpose, a classTimer was made, from the code made freely available by The Cherno on YouTube. (https://www.youtube.com/watch?v=YG4jexlSAjc) In short, the class, when instansiated, records the time it has been created. When it is destroyed, it records the time of destruction, and prints the difference between the two values.

By using C++'s scopes, we can ensure the destruction of the Timer class. An example of how the code was run can be seen below:

```
std::cout << "Running breadth" << std::endl;
{
    Eng::Timer t(fileName);
    Maze.RunBreadth();
}
```

The results generated are all posted below in miliseconds.

Table 1: Runtime results for Test 1 in ms

ms | size	**100x100**	**200x200**	**300x300**	**400x400**	**500x500**
Breadth	9,47	35,65	108,596	185,629	340,536
Dijkstra	9,878	45,386	122,073	314,446	519,384
A*	1,278	6,2	6,47	8,644	13,42
HPA* Total	36,492	156,994	365,937	648,248	1007,88
HPA* Pre	31,768	136,773	310,022	554,337	860,846
HPA* Path	0,747	2,73	6,791	13,968	22,53
JPS	1,109	3,976	6,35	11,505	17,556

Table 2: Runtime results for Test 2 in ms

ms | size	**100x100**	**200x200**	**300x300**	**400x400**
Breadth	3,893	13,193	43,072	90,116
Dijkstra	4,613	12,708	43,812	87,662
A*	3,643	12,916	40,947	85,287
HPA* Total	95,391	405,843	1043,540	1711,02
HPA* Pre	86,695	375,396	952,592	1528,76
HPA* Path	2,257	6,978	38,859	90,533
JPS	4,313	13,743	38,195	88,05

Table 3: Runtime results for Test 3 in ms

ms | size	**100x100**	**200x200**	**300x300**	**400x400**	**500x500**
Breadth	8,367	31,95	88,324	170,764	277,225
Dijkstra	9,215	38,117	85,679	192,761	330,626
A*	8,061	40,333	95,977	219,193	317,188
HPA* Total	135,641	576,565	1132,34	1920,52	3141,33
HPA* Pre	126,425	520,897	1020,96	1698,01	2770,2
HPA* Path	3,397	22,218	56,875	124,821	193,844
JPS	2,489	6,634	14,209	30,255	33,995

Table 4: Runtime results for Test 4 in ms

ms | size	**100x100**	**200x200**	**300x300**	**400x400**	**500x500**
Breadth	9,382	36,369	97,943	200,119	352,934
Dijkstra	9,793	41,373	107,378	230,881	445,387
A*	4,201	15,815	34,292	64,737	96,229
HPA* Total	307,059	1257,87	3004,07	5340,12	8279,67
HPA* Pre	274,139	1097,94	2637,62	4646,15	7115,64
HPA* Path	9,526	50,865	124,951	234,919	455,533
JPS	4,502	13,990	31,716	54,186	90,142

Table 5: Runtime results for Test 5 in ms

ms | size	**100x100**	**200x200**	**300x300**	**400x400**	**500x500**
Breadth	5,011	19,558	47,912	99,403	174,25
Dijkstra	5,43	21,143	53,415	104,046	191,73
A*	4,15	17,38	48,857	75,084	118,748
HPA* Total	36,476	161,331	388,496	709,155	1112,42
HPA* Pre	28,785	130,256	313,599	575,421	871,384
HPA* Path	3,327	12,816	32,144	69,7	128,824
JPS	3,671	13,359	33,266	71,216	119,793

B.2 Memory

These results were gained by exporting the project to XCode. From XCode, the code was modified slighty, to take an integer as input from the terminal, and run the test corresponding to said integer.

```
int i;

std::cout << "Type a number:";
std::cin >> i;
std::cout << std::endl;

std::string fileName = tests[i];
std::string path = "../../tests/" + fileName + ".png";

CImg<unsigned char> * img  = new CImg<unsigned char>(path.c_str());
Eng::Maze Maze(img, fileName);

Maze.RunningTests();

std::cout << "Running <NAME OF ALGORITHM>" << std::endl;
{
    Maze.RunALGORITHM();
}
std::cout << "Done" << std::endl;
```

A breakpoint was then placed at the last line, at which the highest memory used by the program was recorded, provided by the XCode debugger. This test was then run seperately for each test for each algorithm, to make sure, that no rogue memory was present when running a speicifc test.

The results can be seen below in MB.

Table 6: Memory results for Test 1 in MB

MB | size	**100x100**	**200x200**	**300x300**	**400x400**	**500x500**
Breadth	9,3	13	14,7	21,3	25,1
Dijkstra	9,3	12,1	14,5	26,2	28,4
A*	8	8,4	9	9,4	9,8
HPA*	9,6	11,2	12,9	24,9	30,3
JPS	8	8,2	8,9	9,8	10,4

Table 7: Memory results for Test 2 in MB

MB | size	**100x100**	**200x200**	**300x300**	**400x400**
Breadth	8,4	9,7	12,9	18,3
Dijkstra	8,4	9,6	12,5	18,1
A*	8,4	9,5	12,9	16,1
HPA*	9,6	11,6	14,4	21
JPS	8,3	9,5	11,9	16,2

Table 8: Memory results for Test 3 in MB

MB | size	**100x100**	**200x200**	**300x300**	**400x400**	**500x500**
Breadth	9	12,7	16,5	23,2	31,5
Dijkstra	9	12	14,8	26,2	29,3
A*	8,8	12	14,7	20,2	23,1
HPA*	10,1	12,1	16,1	25,7	72,7
JPS	8,2	9,2	10,4	11,3	13,4

Table 9: Memory results for Test 4 in MB

MB | size	**100x100**	**200x200**	**300x300**	**400x400**	**500x500**
Breadth	9,1	12,3	17,600	18,3	25,7
Dijkstra	9,1	11,4	15,6	25	26,5
A*	8,3	9,2	11	13,5	14,7
HPA*	12,4	24,4	55,5	175,5	268,9
JPS	8,3	9,3	10,8	13,7	14,9

Table 10: Memory results for Test 5 in MB

MB | size	**100x100**	**200x200**	**300x300**	**400x400**	**500x500**
Breadth	8,6	10,6	12,2	17,6	21,4
Dijkstra	8,6	10,6	12,4	16	19,4
A*	8,3	9,6	12,4	14,2	16,4
HPA*	9,3	10,5	11,9	17,8	25,5
JPS	8,4	9,9	11,6	14,7	16,4

B.3 Path lengths

As mentioned earlier, moving horizontally or vertically costs 1. Moving diagonally costs $\sqrt{2}$. The results below show the total amount of pixels that had to be traversed, acording to the lengths above.

If a path deviates from the other algorithms, it is shown with red text.

Table 11: Path lengths for test 1

Pixels | size	**100x100**	**200x200**	**300x300**	**400x400**	**500x500**
Breadth	140,007	281,428	422,850	564,271	705,693
Dijkstra	140,007	281,428	422,850	564,271	705,693
A*	140,007	281,428	422,850	564,271	705,693
HPA*	145,279	292,558	439,838	587,117	734,396
JPS	140,007	281,428	422,850	564,271	705,693

Table 12: Path lengths for test 2

Pixels | size	**100x100**	**200x200**	**300x300**	**400x400**
Breadth	859,205	2214,280	4198,550	8376,090
Dijkstra	859,205	2214,280	4198,550	8376,090
A*	859,205	2214,280	4198,550	8376,090
HPA*	894,938	2302,150	4378,390	8738,100
JPS	859,205	2214,280	4198,550	8376,090

Table 13: Path lengths for test 3

Pixels | size	**100x100**	**200x200**	**300x300**	**400x400**	**500x500**
Breadth	276,137	650,558	819,423	1890,880	1223,380
Dijkstra	276,137	650,558	819,423	1890,880	1223,380
A*	276,137	650,558	819,423	1890,880	1223,380
HPA*	301,309	681,487	872,051	1993,330	1292,860
JPS	276,137	650,558	819,423	1890,880	1223,380

Table 14: Path lengths for test 4

Pixels | size	**100x100**	**200x200**	**300x300**	**400x400**	**500x500**
Breadth	146,693	290,215	435,737	583,845	727,367
Dijkstra	145,865	290,215	435,737	583,016	727,367
A*	145,865	290,215	435,737	583,016	727,367
HPA*	152,309	304,274	457,411	610,548	767,200
JPS	145,865	290,215	435,737	583,016	727,367

Table 15: Path lengths for test 5

Pixels | size	**100x100**	**200x200**	**300x300**	**400x400**	**500x500**
Breadth	162,853	327,706	492,558	657,411	822,264
Dijkstra	162,853	327,706	492,558	657,411	822,264
A*	162,853	327,706	492,558	657,411	822,264
HPA*	182,770	371,054	559,338	747,622	935,907
JPS	162,853	327,706	492,558	657,411	822,264

C Code

All the code is freely available to be examined in the GitHub repository at `https://github.com/tikki100/SPBachelor/`. (*Shortest Path Bachelor Code Github* 2020).